

# Mitigating ML-Driven Adversarial Attacks on xApps Using Dynamic Defense Mechanisms

PRUDHVI KUMAR KAKANI<sup>1</sup>, MOHAMMAD ASIF HABIBI<sup>1</sup>,  
MANJUNATH REDDY CHAVVA BALANNAGARI<sup>1</sup>, XAVIER COSTA-PÉREZ<sup>1,2,3,4</sup> (Senior Member, IEEE),  
AND HANS D. SCHOTTEN<sup>1,5</sup> (Member, IEEE)

<sup>1</sup>Division of Wireless Communications and Radio Navigation, Department of Electrical and Computer Engineering, University of Kaiserslautern-Landau, 67663 Kaiserslautern, Germany

<sup>2</sup>6G Networks Research and Development Department, NEC Laboratories Europe, 69115 Heidelberg, Germany

<sup>3</sup>Network Engineering Department, Universitat Politècnica de Catalunya, 08034 Barcelona, Spain

<sup>4</sup>i2CAT Research Center, Catalan Institution for Research and Advanced Studies, 08034 Barcelona, Spain

<sup>5</sup>Intelligent Networking Research Group, German Research Center for Artificial Intelligence, 67663 Kaiserslautern, Germany

CORRESPONDING AUTHOR: M. A. HABIBI (e-mail: asif@eit.uni-kl.de).

This work was supported in part by the German Federal Ministry for Education and Research (BMBF) through the "Open6GHub" project under Grant 16KISK004.

**ABSTRACT** The open radio access network architecture (O-RAN) architecture leverages intelligent near-real-time applications, known as xApps, to optimize network performance and services. However, these machine learning (ML)-driven xApps are vulnerable to adversarial attacks that can compromise their functionality and reliability. In this paper, we present a comprehensive study of adversarial threats targeting xApps and explore dynamic defense mechanisms to mitigate these risks. We begin by identifying potential attack vectors that target the near-real-time RAN intelligent controller (Near-RT RIC) and its associated xApps. We then utilize an open-source O-RAN testbed to deploy a key performance indicator (KPI) Monitoring xApp, a detection xApp, and a malicious adversarial xApp. The adversarial xApp carries out sophisticated inference-time attacks, including Carlini & Wagner (C&W) and basic iterative method (BIM), by perturbing key performance metrics in real-time to mislead the ML-based detection xApp. To counter these threats, we develop a defense xApp that integrates sequential anomaly detection techniques, ensemble deep neural network (DNN) inference, and gradient-based heuristics for real-time attack mitigation. Experimental results demonstrate that the C&W attack significantly degrades the baseline detection performance of the target xApp, reducing its accuracy from 92% to just 16% and BIM attack also achieves a comparable impact, lowering the detection accuracy to around 10%. Nevertheless, the proposed defense xApp promptly detects and neutralizes these adversarial manipulations, thereby restoring the effectiveness of the detector achieving up to 84% accuracy under the (C&W) attack and improving BIM (Basic Iterative Method) attack detection accuracy to 93% in the most challenging scenarios. This work presents a closed-loop evaluation of adversarial attacks and corresponding defenses within a real-world O-RAN environment that provides valuable insights into real-world vulnerabilities and mitigation strategies. By introducing a dynamic defense framework, we significantly enhance the security and resilience of ML-driven xApps and maintain reliable O-RAN performance even during attacks.

**INDEX TERMS** Adversarial attacks, artificial intelligence, basic iterative method, Carlini & Wagner method, defense mechanisms, machine learning, Near-RT RIC, O-RAN, privacy, security, xApps.

## I. INTRODUCTORY REMARKS

THE open radio access network (O-RAN) architecture, standardized by the O-RAN Alliance, is a revolutionary approach that is transforming the wireless communication

industry [1]. By eliminating the limitations of proprietary systems, O-RAN enables a new era of interoperability and programmability, where hardware and software from different vendors can efficiently and intelligently interact

and interoperate [2], [3]. This innovative approach enables network operators to build flexible, interoperable cellular networks by integrating components from multiple vendors within a common wireless infrastructure [1], [4]. The openness feature of the O-RAN architecture promotes healthy competition [5], innovation, and cost efficiency. Therefore, wireless operators become more agile and responsive, adapting rapidly to the ever-evolving wireless communication systems [6].

The O-RAN architecture extends the 3rd Generation Partnership Project (3GPP)-defined next-generation radio access network (NG-RAN) architecture by introducing a more disaggregated and flexible approach to state-of-the-art RANs [2]. Building upon the 3GPP's functional split of the next generation node B (gNB), the O-RAN Alliance further divides an open-next generation node B (O-gNB) into three major components: open centralized unit (O-CU), open-distributed unit (O-DU), and open-radio unit (O-RU). One of the innovative components of O-RAN is the RAN intelligent controller (RIC), which comes in two variants [1]: the non-real-time RAN intelligent controller (Non-RT RIC) and the near-real-time RAN intelligent controller (Near-RT RIC). These two RICs abstract RAN control and monitoring functions, as well as enable advanced automation and intelligence [1], [2]. Both RICs support the integration of third-party applications known as rApps and xApps, which play a crucial role in enhancing network performance and security. This design, complemented by the service management and orchestration (SMO) framework and the open cloud (O-Cloud) for hosting virtual network functions (VNFs) [7], [8], promotes multivendor interoperability and scalable deployments through open, programmable, and standardized (or standard-compliant) interfaces [9].

Focusing on the Near-RT RIC in this article, it is an intelligent component that controls and optimises RAN functions with latencies between 10 milliseconds to 1 second [10], enabling efficient network optimization and intelligent decision-making in O-RAN. xApps are microservice-based, programmable applications that operate on the Near-RT RIC and can be developed and deployed by network operators and authorized third-party vendors [10]. These xApps can leverage machine learning (ML) algorithms to analyze network data in real-time and are utilized for predicting and mitigating security threats to O-RAN and its components [8], [11]. Within the Near-RT RIC, the shared data layer (SDL) is a centralized data management framework that provides xApps with access to real-time and historical network data from various databases [12]. SDL ensures efficient data sharing and consistency across multiple xApps, enabling them to work collaboratively without redundant data retrieval. A database within the Near-RT RIC supports both the SDL and xApps by storing structured and unstructured network data. It serves as a high-performance backend for real-time decision-making and data analysis [12]. The security frameworks of the Near-RT RIC and xApps can be enhanced by the integration of artificial intelligence (AI)/ML capabilities [4],

particularly in protecting critical components associated with them, such as the E2 interface and databases [13].

The AI/ML models deployed within xApps can analyze a large amount of network data in real-time that enable detection and mitigation of security threats [14]. These models can identify anomalous behavior patterns in network traffic that may predict ongoing attacks or unauthorized access attempts. The Near-RT RIC, with its millisecond-level control execution, enables immediate implementation of security measures based on AI/ML analysis [11], [14]. However, the integration of AI/ML also introduces new security challenges [4], [15], such as adversaries that attempt to manipulate ML models, which alter network behavior or evade detection mechanisms [11], [14]. To counter these threats, ongoing research focuses on developing robust security frameworks for AI/ML within the Near-RT RIC, including methods to defend against adversarial attacks and ensure data integrity for training and inference. The O-RAN Security Working Group emphasizes securing AI/ML across O-RAN to ensure that these technologies enhance network security while remaining resilient against emerging threats [16], [17].

#### A. LITERATURE REVIEW

The O-RAN Alliance has recently explored the integration of AI/ML-assisted solutions into the Near-RT RIC and xApps to enhance real-time network optimization and security. Three of its documents provide a comprehensive foundation for understanding the role of AI/ML in securing and optimizing the Near-RT RIC and xApps [12], [14], [16]. They highlight the importance of AI/ML-driven real-time decision-making, anomaly detection, and network optimization while addressing security challenges such as adversarial attacks, data manipulation, and vulnerabilities in xApps [18]. The three documents mostly incorporate AI/ML models with a centralized approach in which databases facilitate real-time data processing and intelligent responses. However, this approach faces two main challenges. First, despite the existence of a standardized AI/ML framework, there are ongoing issues related to flexibility, security, and interoperability across O-RAN components and interfaces, which require rigorous testing methodologies to ensure consistency in model training and prediction. Second, the integration of AI/ML models introduces potential security vulnerabilities, including vulnerability to adversarial attacks and data poisoning, which require mitigation strategies to ensure the trustworthiness and resilience of AI-driven functionalities within the Near-RT RIC and xApps [13], [19].

In addition to the O-RAN Alliance, several researchers contributed to the integration of the AI/ML capabilities within the Near-RT RIC and xApps. For instance, authors in [20] identified security vulnerabilities in an O-RAN H-Release environment due to inadequate access control for xApps, enabling potential attacks on application programming interfaces (APIs) and RAN services. They presented three attack scenarios and reported the threats to common

vulnerabilities and exposures. Moreover, [21] developed an interference classifier xApp using a deep neural network (DNN) methodology within the Near-RT RIC and tested its vulnerability to adversarial attacks. Furthermore, paper [22] presented an AI/ML-assisted security approach with two xApps: intrusion detection system (IDS) xApp and secure slicing (SS) xApp. The IDS xApp uses a self attention enhanced recursive neural network (RNN) autoencoder for anomaly detection. The SS xApp is utilized for dynamic threat mitigation through network slicing. This approach ensures real-time intrusion detection by using IDS xApp and secure communication by isolating malicious users with SS xApp. Finally, article [23] introduces a Zero Trust security framework for radio access network (RAN) designed by leveraging xApps, such as service authentication, intrusion detection, and secure slicing xApps. The authentication xApp implements multi-factor authentication (MFA) to ensure secure user verification, while the intrusion detection xApp continuously monitors network traffic, utilizing behavior analysis on real-time data to detect anomalies. Upon identifying a potential threat, the secure slicing xApp dynamically reallocates network resources to isolate the malicious user, thereby maintaining secure and efficient communication for legitimate users. The authors implemented this security framework on open-source platforms and validated it through experimental evaluation.

### B. MOTIVATION AND RESEARCH CHALLENGE

In addition to the various research studies from academia and the standardization efforts by the O-RAN Alliance, a major challenge in implementing AI/ML capabilities within xApps for security purposes on open-source testbeds is **their limited ability to handle multiple base stations and a large number of user equipments (UEs)**. Furthermore, the key performance indicator (KPI) Monitoring xApp in the open source O-RAN Near-RT RIC, which is the main focus of our research, **supports only a limited set of KPI parameters**. To fully exploit AI/ML driven network optimization, it is essential to expand the range of KPIs to include a broader set of performance metrics. However, extracting these additional KPIs through the KPI Monitoring xApp is a significant challenge. Moreover, the open-source nature of these testbeds complicates the validation of AI/ML models under real world network conditions. **The absence of real-world traffic patterns, interference, and user behavior further complicates data collection and labeling**. This challenge limits the development and evaluation of accurate AI/ML-based network optimization solutions.

Furthermore, **the complexity and diversity of data from various network components used for model training** introduce significant challenges in integrating AI/ML models into the Near-RT RIC and xApps. The interpretation, normalization, and processing of this data are critical tasks that require efficient AI/ML solutions to manage the workflow effectively. Developing xApps capable of managing data flow and ensuring that **AI/ML models are trained,**

**updated, and deployed efficiently remains a significant research challenge**. Addressing these challenges is crucial for unlocking the full potential of AI/ML algorithms within the Near-RT RIC and xApps. Finally, a robust AI/ML-assisted framework within the O-RAN architecture can improve performance, optimize resources, and strengthen security.

### C. MAJOR CONTRIBUTIONS AND GOALS

In addressing the identified research challenges, this paper makes the following key contributions:

- First, we conduct a **comprehensive investigation of cybersecurity threats by reviewing O-RAN Alliance specifications**, with a particular focus on adversarial attacks targeting the xApps. Through this review, we identify key issues that require further exploration.
- Second, we focus on **an experimental investigation using a real-time open source O-RAN deployment** to examine the impact of adversarial attacks on network performance within the Near-RT RIC and xApps.
- Third, we redesign a **KPI Monitoring xApp to capture a broader range of KPI parameters**, as the current corresponding xApp is limited to a few parameters. We believe that expanding the KPI set is essential for comprehensive network performance assessment and enhancing AI/ML-driven network optimization.
- Fourth, we integrate **AI/ML models into our xApp** to analyze network data effectively. This integration enables real-time processing and decision-making, which optimizes network performance through intelligent insights derived from the network data.
- Fifth, we perform **various adversarial attacks by using a compromised xApp** to manipulate data within the databases of the Near-RT RIC. These attacks aim to exploit vulnerabilities in the system by altering the data used by the AI/ML models that potentially compromise the integrity of network operations.
- Sixth, we implement **mitigation techniques by developing a defense xApp that uses AI/ML** to mitigate the adversarial attacks and maintain network stability.

### D. THE STRUCTURE OF THE ARTICLE

The remainder of this paper is structured as follows. Section II provides an overview of the O-RAN architecture, describing its core components and functionalities, and highlighting specific security challenges associated with ML-driven applications. Section III explores adversarial attack models designed to compromise the integrity of data within databases used by ML models in the Near-RT RIC and xApps. Section IV examines various adversarial attack methodologies, with a focus on how these threats exploit vulnerabilities in ML systems and their potential impact on network operations. Section V outlines the experimental setup and the practical implementation of both adversarial attacks and corresponding defense mechanisms. Section VI

presents a comprehensive evaluation of the experimental results, including performance metrics and a comparative analysis of mitigation strategies. Finally, Section VII summarizes the main contributions of this work and outlines future research directions.

## II. UNDERSTANDING THE NEAR-RT RIC ARCHITECTURE, SECURITY CHALLENGES, AND DEFENSE STRATEGIES

In this section, we examine the role of the Near-RT RIC and provide an overview of its key components and functionalities. We specifically highlight the Near-RT RIC's critical function in enabling intelligent, dynamic control of O-RAN operations through xApps. In addition, we present a detailed analysis of ML-based attacks, including adversarial threats. The section concludes by exploring various defense mechanisms, with a particular focus on AI-based security solutions designed to mitigate adversarial attacks in O-RAN.

### A. THE NEAR-RT RIC PLATFORM AND XAPPS

Within O-RAN, the Near-RT RIC platform and xApps are expected to leverage intelligent and autonomous solutions to address various optimization challenges, including security and privacy, on a large scale. To fully understand the application of AI/ML solutions in enhancing O-RAN security and privacy, we begin by discussing the key functionalities of the Near-RT RIC platform and xApps in this subsection.

#### 1) NEAR-RT RIC PLATFORM

The Near-RT RIC is designed to enhance network optimization and improve system performance within a control loop that operates at periodicities ranging from  $\geq 10$  milliseconds to  $< 1$  second [10]. The Near-RT RIC architecture consists of the Near-RT RIC platform and the xApps [8].

The Near-RT RIC platform is responsible for hosting, managing, and orchestrating xApps to facilitate near-real-time network optimization. It supports the efficient deployment of xApps through lifecycle management, monitoring, upgrading, and scalability. Through the E2 interface, the Near-RT RIC platform communicates with E2 nodes, namely the O-CU and O-DU. This interaction facilitates near-real-time network optimization and control of these nodes [24].

To support intelligent decision-making, the Near-RT RIC platform provides data management services that collect, store, and distribute network telemetry, performance metrics, and control inputs [9]. Furthermore, it interacts with the Non-RT RIC through the A1 interface to receive trained AI/ML models and A1 policy guidance [24]. These optimization solutions enable network operators to dynamically adjust network behavior through near-real-time data analysis.

In addition, the Near-RT RIC provides an execution environment for xApps with standardized APIs to ensure seamless integration and communication among multiple

xApps. By incorporating AI/ML-driven decision-making, real-time analytics, robust security, and scalable xApp orchestration, the platform optimizes network operations. This combination enhances network efficiency, reduces interference, and improves the overall performance of O-RAN [12].

#### 2) XAPPS

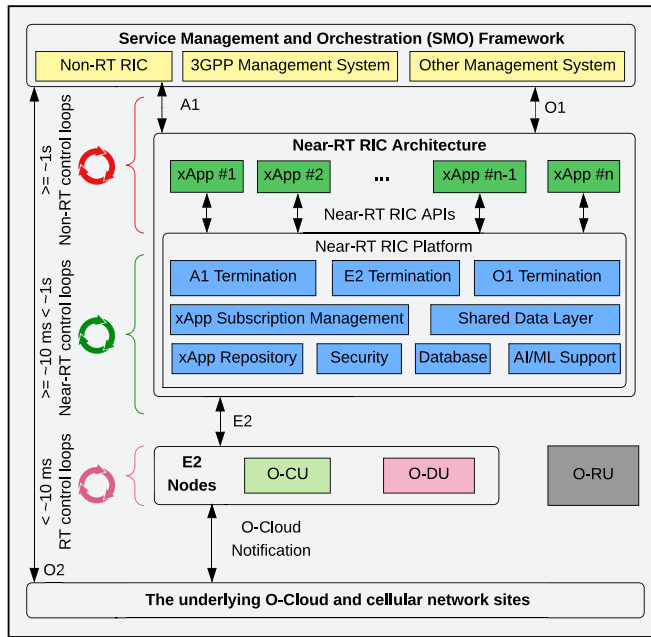
xApps are customized software applications designed to operate within Near-RT RIC. Their primary objective is to optimize various functionalities, such as mobility management, load balancing, interference detection, and others [12]. These applications function within the near-real-time control loop of the RIC and enable dynamic network adjustments through the E2 interface with O-CUs and O-DUs (see Figure 1). Through the E2 interface, xApps interact with O-CU and O-DU to collect data, analyze network conditions, and implement control actions. During the establishment of the E2 interface connection, xApps subscribe to event-triggered information from O-gNBs, which expose their supported functionalities using E2 service model (E2SM) [20].

Some xApps leverage AI/ML models to analyze real-time RAN data and implement adaptive optimization strategies. This approach enhances the intelligence and efficiency of the Near-RT RIC. These applications employ various ML techniques, including DNNs and reinforcement learning, to dynamically optimize network parameters. For instance, ML-driven xApps can perform tasks such as traffic prediction, anomaly detection, and intelligent resource allocation [19].

The integration of AI/ML within xApps enables more sophisticated decision-making, which facilitates proactive network management and improved quality of service (QoS). It also strengthens network security through advanced threat detection. However, these models also introduce new vulnerabilities, such as adversarial attacks that can misclassify network events or disrupt resource allocation [25]. To mitigate these risks, extensive research focuses on developing robust ML algorithms. Among various approaches, federated learning is widely adopted to enhance privacy and establish secure deployment frameworks within O-RAN. Ultimately, balancing AI-driven capabilities with the security of xApps remains a critical research challenge in O-RAN [21].

#### 3) SHARED DATA LAYER AND DATABASE

The SDL and Database are essential functionalities within the Near-RT RIC platform, as shown in Figure 1. They play a critical role in data management, storage, and exchange. Specifically, the SDL serves as a standardized data access interface, enabling xApps to subscribe to database notifications [13]. Real-time notifications include performance metrics, network state changes, event triggers, and alarms. The SDL also allows xApps to perform read and write operations on O-RAN and UE data. By providing a centralized data repository, it facilitates efficient information exchange, enhancing communication among network functions [12].



**FIGURE 1.** A high-level overview of the O-RAN architecture with a particular focus on Near-RT RIC, the role of xApps, and surrounding functional entities.

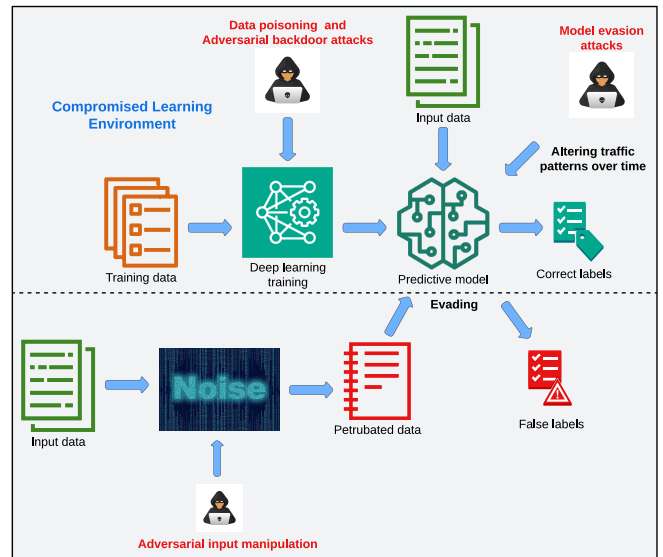
The Database provides persistent storage for both structured and unstructured data. It stores network-related and UE-specific information, including details of UEs associated with E2 nodes, xApps, and their mappings. By ensuring data integrity, scalability, and network intelligence, the Database facilitates coordination among various Near-RT RIC functionalities and xApps. This capability supports robust AI/ML-driven decision-making, significantly enhancing the overall operational efficiency and maintenance of the entire Near-RT RIC architecture [9].

#### 4) OTHER KEY FUNCTIONALITIES OF THE NEAR-RT RIC PLATFORM

Figure 1 shows that the Near-RT RIC platform comprises several functionalities. In this subsection, we discuss only those functionalities that are referenced in this paper.

The E2 Termination is responsible for handling the E2 interface. The goal of this termination is to ensure an efficient and standard-compliant communication between the Near-RT RIC and E2 nodes [9]. Acting as a logical endpoint, it manages E2 signaling, subscription handling, real-time data collection and reporting, and policy enforcement within the Near-RT RIC [26] platform.

The Security functionality is responsible for protecting operations and sensitive data from potential security threats. While significant progress has been made in developing security mechanisms for the Near-RT RIC platform, further efforts are needed to fully standardize zero-trust security architectures. Additionally, improvements are required in anomaly detection mechanisms, the implementation of real-time threat mitigation strategies, and the enhancement of AI/ML-driven security operations [11].



**FIGURE 2.** Classification of adversarial attacks on deep learning models based on their phase of occurrence.

The xApp Repository is a critical functionality that acts as a centralized platform for storing, managing, and deploying xApps. It ensures the seamless integration and interoperability of xApps within Near-RT RIC [12].

The AI/ML functionality plays a vital role in network automation by enabling the analysis of extensive O-RAN datasets, predicting network behavior, and continuously optimizing decision-making processes. By leveraging advanced AI/ML models, this functionality enhances real-time adaptability, improves resource allocation, and strengthens the overall efficiency of the Near-RT RIC architecture [16].

#### B. ML-BASED ADVERSARIAL SECURITY ATTACKS

The increased integration of ML algorithms introduces vulnerabilities to sophisticated adversarial attacks [14], [27], which can potentially compromise the performance and security of the Near-RT RIC platform and its xApps. In this subsection, we discuss a set of key adversarial security threats that specifically target ML-based applications and optimization solutions [14]. As illustrated in Figure 2, the training phase occurs within a compromised learning environment. During this phase, various adversarial attacks including data poisoning, adversarial backdoor attacks, and model evasion attacks are introduced. Once deployed, the model is vulnerable to adversarial input manipulation, potentially diminishing its predictive performance. This vulnerability is quantitatively assessed through the comparative analysis presented in Table 1.

##### 1) ADVERSARIAL INPUT MANIPULATION

In these types of attacks, adversaries introduce small, deliberate changes to the input data in order to deceive ML models into making incorrect decisions [28]. Techniques such as the Fast Gradient Sign Method (FGSM) and

**TABLE 1.** The Comparison of the ML-centric attack vectors within Near-RT RIC.

| Attack Type                    | Attack Phase        | Method                                                | Impact on ML Model                                                 | Detection Difficulty |
|--------------------------------|---------------------|-------------------------------------------------------|--------------------------------------------------------------------|----------------------|
| Adversarial Input Manipulation | Inference           | Small, deliberate perturbations to input              | Misclassification of live inputs (e.g., interference mis-labeling) | Moderate             |
| Data Poisoning Attacks         | Training            | Injecting malicious data into training dataset        | Corrupted behavior and inaccurate predictions                      | High                 |
| Model Evasion Attacks          | Inference (Gradual) | Gradually altering input patterns to bypass detection | Stealthy bypass of anomaly detection models                        | High                 |
| Adversarial Backdoor Attacks   | Training            | Embedding hidden triggers in training data            | Trigger-specific incorrect behavior or data leakage                | Very High            |

projected gradient descent (PGD) are commonly employed to exploit vulnerabilities in ML models that lead to misclassifications [27], [29]. These adversarial manipulations can cause legitimate xApps to incorrectly classify interference patterns, thereby triggering inappropriate responses from the Near-RT RIC or associated xApps. As a result, the overall service quality and capacity of the Near-RT RIC architecture may be significantly degraded [16], [28].

## 2) DATA POISONING ATTACKS

The data poisoning attacks target the integrity of training datasets by injecting malicious data into the learning process of the ML models [16]. For instance, a compromised xApp may manipulate the training data used by an ML model responsible for predicting user demand or managing resource allocation. Such tampering can lead to inaccurate predictions, thereby degrading the performance and efficiency of the entire Near-RT RIC architecture [12].

## 3) MODEL EVASION ATTACKS

In such attacks, attackers may gradually manipulate the behavior of ML models for example, by subtly altering traffic patterns over time to degrade the accuracy of detection mechanisms. These attacks exploit models that rely on identifying consistent deviations from normal patterns, allowing malicious activities to remain undetected. For instance, if an xApp is designed to detect anomalous traffic indicative of a distributed-denial-of-service (DDoS) attack, an attacker could adapt their traffic patterns to avoid triggering alarms. This strategy ultimately compromises the availability and integrity of the Near-RT RIC architecture [14], [16].

## 4) ADVERSARIAL BACKDOOR ATTACKS

These adversarial backdoor attacks represent a sophisticated method of compromising ML models. In this approach, attackers introduce hidden vulnerabilities or activation mechanisms, commonly known as backdoors, into ML models during the training phase. For instance, an attacker could implant a backdoor trigger within an xApp responsible for resource allocation. When exposed to specific input patterns, the compromised model may misallocate critical resources or leak sensitive data. Therefore, adversarial backdoor attacks

can undermine the security and operational stability of the xApps and the Near-RT RIC platform [16].

The four aforementioned adversarial attacks pose significant threats to the integrity and reliability of ML-based applications within Near-RT RIC. These challenges highlight the critical need for robust security solutions to defend against such adversarial threats. In the following subsection, we discuss potential strategies to mitigate these attacks.

## C. SOLUTIONS FOR ML-BASED ADVERSARIAL SECURITY ATTACKS

To effectively mitigate major adversarial security attacks targeting ML models within the Near-RT RIC architecture, tailored strategies can be adopted for each attack type. In the remainder of this subsection, we outline several mitigation approaches corresponding to the aforementioned attacks.

### 1) MITIGATIONS FOR ADVERSARIAL INPUT MANIPULATION

- *Input Validation and Sanitization:* This strategy involves implementing robust validation mechanisms to ensure that all incoming data are checked for anomalies or malicious alterations before being processed by ML models. Techniques may include statistical analysis or outlier detection methods [16].
- *Adversarial Training:* This approach enhances the resilience of ML models by incorporating adversarial examples during the training phase. It involves generating manipulated inputs and integrating them into the training dataset to improve model robustness.
- *Ensemble Methods:* This mitigation strategy leverages ensemble learning techniques that aggregate predictions from multiple ML models, thereby reducing the likelihood that adversarial inputs will successfully deceive all ML models simultaneously.

### 2) MITIGATIONS FOR DATA POISONING ATTACKS

- *Data Integrity Checks:* This strategy employs checksums and cryptographic signatures to verify the integrity and authenticity of training datasets. Such mechanisms help detect unauthorized modifications before the data are used to train ML models within xApps.

**TABLE 2.** Comparison of defense strategies against key adversarial attacks on ML models.

| Attack Type                    | Mitigation Focus                | Key Techniques                                               | Objective                                              |
|--------------------------------|---------------------------------|--------------------------------------------------------------|--------------------------------------------------------|
| Adversarial Input Manipulation | Input sanitization & robustness | Input validation, adversarial training, ensemble learning    | Enhance model resilience to adversarial examples       |
| Data Poisoning Attacks         | Data Integrity & Auditing       | Checksums, anomaly detection, regular audits                 | Prevent model corruption via compromised training data |
| Model Evasion Attacks          | Dynamic Detection & Monitoring  | Robustness testing, adaptive thresholds, behavioral analysis | Sustain detection under evolving attack patterns       |
| Adversarial Backdoor Attacks   | Model Integrity & Verification  | Model verification, secure training, output monitoring       | Detect and neutralize hidden triggers in the model     |

- *Anomaly Detection in Training Data:* This approach utilizes anomaly detection algorithms to identify and eliminate suspicious or atypical data points in the training dataset, which may indicate poisoning attempts.
- *Regular Audits of Training Data:* This strategy involves conducting periodic audits of the training data to uncover irregularities or patterns that could suggest the presence of poisoning attacks.

### 3) MITIGATIONS FOR MODEL EVASION ATTACKS

- *Robustness Testing:* This strategy involves regularly evaluating the resilience of ML models against various evasion techniques. Such testing helps uncover vulnerabilities that require further mitigation.
- *Adaptive Security Mechanisms:* This approach deploys adaptive security measures that can dynamically adjust model parameters or detection thresholds based on observed traffic patterns, thereby complicating attempts to bypass detection systems.
- *Behavioral Analysis:* This strategy utilizes behavioral analysis tools to continuously monitor traffic flows and user activities, facilitating the detection of anomalies that may indicate evasion attempts.

### 4) MITIGATIONS FOR ADVERSARIAL BACKDOOR ATTACKS

- *Model Verification Techniques:* This strategy applies verification techniques to systematically examine the behavior of ML models under diverse inputs. It process helps to identify potential backdoor triggers and ensures the absence of hidden malicious functionalities [16].
- *Secure Model Training Environments:* This strategy conducts model training within secure, access-controlled environments, ensuring that only data from verified and trusted sources are utilized.
- *Continuous Monitoring of Model Outputs:* This strategy implements continuous monitoring of model outputs in production to detect unexpected behaviors or responses that could indicate backdoor activities.

By implementing these targeted mitigation strategies, operators can enhance the resilience of ML-based applications within the Near-RT RIC against adversarial attacks, ensuring more secure and reliable operations within O-RAN.

## III. ADVERSARIAL ATTACK MODEL

The process of traditional ML can be divided into two primary phases: the training phase and the deployment phase. In the training phase, the process begins with a dataset known as the training data, which comprises input data paired with corresponding correct labels or expected output values. This data is fed into a deep learning model, a type of ML algorithm capable of automatically learning patterns and representations from the data. During the training, the model iteratively adjusts its internal parameters to minimize errors between its predictions and the actual labels. After successful training, the deep learning model becomes a predictive model, capable of generating accurate predictions when exposed to new, unseen input data. Ideally, these predictions align closely with the correct labels, demonstrating the model's ability to generalize beyond the training set.

In our system model, we assume an insider threat within the Near-RT RIC, represented by a malicious or compromised xApp that operates with the same feature inputs and shared database access as the targeted xApp. The attacker is capable of actively manipulating input data by intercepting and injecting bounded adversarial perturbations into the input data in the database. We adopt a white box attack model, where the adversary has full knowledge of the target xApp's model architecture and parameters, enabling the generation of targeted and timely perturbations to induce specific misclassifications or decision errors. The attacker is constrained by the need to keep perturbations small and plausible to avoid detection, and is limited by realistic computational capabilities excluding attacks that rely on excessive or impractical computation.

This phenomenon highlights a critical weakness in ML system: their susceptibility to adversarial manipulation. Despite achieving high accuracy on standard data, models can fail spectacularly when exposed to maliciously crafted inputs. Understanding and addressing this vulnerability is essential for developing robust and secure ML systems capable of resisting adversarial attacks and maintaining reliability in real-world applications and use cases. Several adversarial attack methods will be tested in this study to demonstrate how vulnerable the xApps are to such threats. In the following, we explore two methods.

## A. CARLINI AND WAGNER ATTACK

The Carlini–Wagner (C&W) attack is a sophisticated method for generating adversarial examples against neural networks, introduced by Nicholas Carlini and David Wagner in 2017 [30]. This attack is notable for its effectiveness and flexibility, allowing it to target various types of neural network architectures. This attack primarily focuses on generating adversarial examples that are minimally perturbed versions of original inputs, ensuring they remain close to the original data while being misclassified by the model [30].

### 1) OPTIMIZATION FRAMEWORK

The C&W attack formulates an optimization problem that seeks to minimize the distance between the original input and the adversarial example, while simultaneously ensuring that the adversarial example is classified as a specific target class [30]. The objective function is defined as:

$$f(x') = \max \left( \text{logit}(x')_t - \max_{i \neq t} \text{logit}(x')_i, -\kappa \right), \quad (1)$$

where  $\text{logit}(x')_i$  is the logit (pre-softmax output) of class  $i$  for the input  $x'$ ,  $t$  is the target class (for targeted attacks), and  $\kappa$  is the confidence parameter. It is worth noting that higher  $\kappa$  values enforce a larger margin between the target logit and the next highest logit, making the attack stronger.

### 2) NORM CONSTRAINTS

The C&W attack generates adversarial examples under different  $\ell_p$ -norm constraints:

- $\ell_2$ -norm: Minimizes the Euclidean distance between the original input and the adversarial example.
- $\ell_\infty$ -norm: Limits the maximum change applied to any individual feature.
- $\ell_0$ -norm: Limits the number of modified features.

### 3) ITERATIVE APPROACH

In the iterative approach, the attack uses an iterative gradient descent method to optimize the adversarial perturbation. This approach enables the attack to escape local minima, maintain high attack success rates even against defenses, and generate minimally perturbed but effective adversarial examples in complex scenarios [30], [31].

### 4) TARGETED ATTACK

In targeted attacks, the adversarial example  $x'$  is generated such that it is classified as a specific, attacker-chosen target class  $t$  [32].

### 5) UNTARGETED ATTACK

In untargeted attacks, the goal is to cause any misclassification. The adversarial example  $x'$  should not be classified as its true label  $y$ .

### 6) ALGORITHM

In this method, the core steps of the C&W attack are:

- *Input:* Original input  $x$ , true class  $y$ , target model  $F$ , confidence  $\kappa$ , and norm  $p$ .
- *Initialization:* Set the initial perturbation  $\delta = 0$ .
- *Transform Input:* Reparameterize input using a variable  $w$  such that  $x' = \tanh(w)$ , ensuring that  $x'$  lies within valid input bounds.
- *Optimization:*

$$\min \|\delta\|_p + c \cdot f(x + \delta)$$

Update  $w$  using the Adam optimizer to minimize the total loss, which balances perturbation magnitude and attack success.

- *Binary Search:* Iteratively adjust the constant  $c$  to find the minimal perturbation  $\delta$  that achieves successful misclassification.
- *Output:* Return final adversarial example  $x' = x + \delta$ .

## B. BASIC ITERATIVE METHOD ATTACK

The basic iterative method (BIM) is an adversarial attack that incrementally perturbs an input  $x$  in multiple steps to produce an adversarial example  $x'$ . It is an iterative extension of the FGSM [33], [34], designed to improve attack effectiveness by applying the gradient multiple times with smaller step sizes.

### 1) OBJECTIVE OF THE BIM ATTACK

The goal is to maximize the model's loss function  $J(\theta, x', y)$  while ensuring that the perturbation  $\delta$  remains imperceptible. Formally, the BIM attack aims to craft an adversarial example  $x'$  such that:

$$x' = x + \delta \quad \text{subject to} \quad \|\delta\|_\infty \leq \epsilon, \quad (2)$$

where  $\epsilon$  is the perturbation budget that bounds the  $L_\infty$  norm of the perturbation [35].

### 2) BIM ATTACK ALGORITHM

The major steps of the BIM attack algorithm are:

#### Initialization:

- Start with clean input:  $x_0 = x$ .
- Define perturbation budget:  $\epsilon$ .
- Choose step size:  $\alpha$ .
- Set the total number of iterations:  $T = \min(\frac{\epsilon}{\alpha}, T_{\max})$ .

#### Iterative Process:

For each iteration  $t = 1, 2, \dots, T$ :

- Compute gradient:  $g_t = \text{sign}(\nabla_{x_t} J(\theta, x_t, y))$ .
- Apply perturbation:  $x_{t+1} = x_t + \alpha \cdot g_t$ .
- Clip to enforce constraint:  $x_{t+1} = \text{Clip}_{x, \epsilon}(x_{t+1})$ , where the clipping function is defined as:  $\text{Clip}_{x, \epsilon}(x') = \min(\max(x', x - \epsilon), x + \epsilon)$ .

After  $T$  iterations, final adversarial example  $x_T$  is returned.

### 3) MATHEMATICAL FORMULATION

Iterative update rule for BIM can be compactly written as:

$$x_{t+1} = \text{Clip}_{x,\epsilon}(x_t + \alpha \cdot \text{sign}(\nabla_{x_t} J(\theta, x_t, y))), \quad (3)$$

where  $x$  is original input,  $x_t$  is the perturbation state at iteration  $t$ ,  $J(\theta, x_t, y)$  is model loss function for input  $x_t$  and label  $y$ ,  $\nabla_{x_t} J$  is gradient of the loss with respect to the input,  $\alpha$  is step size per iteration,  $\epsilon$  is maximum perturbation budget, and  $\text{Clip}_{x,\epsilon}$  is projection back into the valid  $\epsilon$ -ball around  $x$ .

## IV. MACHINE LEARNING DEFENSE MECHANISMS

To mitigate the adversarial attacks discussed in the previous section, several defense strategies can be considered. These strategies are discussed in this section in a detailed manner.

### A. GRADIENT NORM THRESHOLDING

This defense mechanism detects adversarial samples by using the internal model gradients. The gradient of the model output concerning the input features—how sensitive the prediction is to slight changes in each input dimension—is calculated by the xApp whenever a new measurement is fed into the model. This gradient vector's norm (magnitude),  $|\nabla_x F(x)|$ , is then computed. The idea is that adversarial inputs frequently cause the model to be positioned in a flat area of the decision surface, which leads to a minimal gradient norm. To put it another way, a well-designed adversarial sample may make the model overconfident in a false prediction, where slight changes in the input have little effect on the output, resulting in an almost zero gradient. This gradient norm has a threshold set by the defense; if the norm is less than the threshold, the input is marked as possibly hostile. Because regular inputs (not adversarially optimized) usually do not land in such extreme flat regions, this serves as a warning sign.

### B. Z-SCORE BASED ANOMALY DETECTION

Statistical outlier detection on per-UE measurements is used in this defense mechanism. Over a historical period, the system continuously calculates a running mean ( $\mu$ ) and standard deviation ( $\sigma$ ) for each UE's feature values. A Z-score is computed as follows for every new measurement:  $Z = \frac{x_{\text{new}} - \mu_{\text{hist}}}{\sigma_{\text{hist}}}$ . The measurement is marked as anomalous if the absolute Z-score is higher than a predetermined threshold. A potential attack or malfunction is identified by standardizing recent UE feature values against their historical statistics and identifying any value that is an extreme outlier compared to the UE's history.

### C. ENSEMBLE PREDICTION CONFIDENCE

The system uses an ensemble approach that uses multiple forward passes to measure prediction confidence in order to improve the robustness of UE measurements against hostile inputs. The model performs multiple forward passes on the same input, using methods such as dropout or

adding a small amount of randomness during inference, rather than depending on a single inference, and then averages the outcomes. For the given UE measurement, this method produces a distribution of predictions from which a consensus prediction and a confidence measure can be obtained.

### D. FEATURE DELTA THRESHOLDING

While the Z-score detection method compares absolute values to historical averages, this defense strategy focuses on abrupt changes in the features reported by a UE. For every key feature, the system calculates the difference (Delta) between two successive measurement reports from the same UE:  $\Delta(x) = x_t - x_{t-1}$ . This change is immediately marked as suspicious if it is substantial and surpasses a predetermined threshold. The fundamental idea is that valid metrics from the UE typically change gradually over time due to network and physical dynamics. Significant discrepancies between reports are therefore regarded as either unusual or suggestive of manipulation. By applying thresholding to the feature deltas, the xApp can detect anomalies that appear as sudden shifts in a brief period of time.

### E. TEMPORAL VOTING WITH HISTORY

To increase stability and lower false alarms, the xApp does not base its suppression decisions on a single detection event. Rather, it employs a voting or majority rule over a brief history of recent detection decisions for every UE. For instance, the system may keep track of each UE's most recent  $N$  anomaly detection results, including whether an attack was detected or not. After updating this history with each new measurement, it looks at the pattern and determines that the UE is being attacked if most of the previous  $N$  readings have been marked as abnormal. On the other hand, the lone outlier is probably disregarded as a temporary fluctuation if only one of the previous  $N$  readings was abnormal and the others were regular. An adversarial condition must endure for several consecutive reports before the UE is suppressed, according to this temporal voting scheme.

## V. IMPLEMENTATION OF ATTACKS AND DEFENSE MECHANISMS ON AN O-RAN TESTBED

The implementation of our proposed adversarial attacks and defense mechanisms was conducted on a modular O-RAN testbed that emulates real-world fifth-generation (5G) network behavior with controllable attack surfaces. This section presents the end-to-end (E2E) deployment and evaluation of xApps within the Near-RT RIC, which covers both benign monitoring and adversarial manipulation scenarios. We describe the software architecture, the KPI data collection pipeline, the design and training of a DNN-based detection system, and the execution of adversarial attacks using the C&W and BIM methods. Furthermore, we demonstrate a real-time, closed-loop workflow that illustrates the interaction of these components under latency-constrained Near-RT RIC conditions.

### A. TESTBED ARCHITECTURE AND DEPLOYMENT

We prototyped our framework on an O-RAN testbed integrating a 5G RAN, a 5G core network, and a containerized Near-RT RIC. The O-RAN provides the O-gNB, consisting of disaggregated O-CU and O-DU components. In our setup, we utilized a universal software radio peripheral (USRP) B210 as an O-RU, enabling a complete RAN (namely the srsRAN) connection between the UE and the 5G core network (namely the Open5GS) for E2E 5G connectivity, as illustrated in Figure 3. In our setup, the RIC platform runs inside a Docker container and is connected to the O-gNB. The Near-RT RIC (namely the O-RAN Software Community Near-Real-Time RAN Intelligent Controller (O-RAN SC RIC)) hosts modular, microservice-based applications are called xApps, which enable intelligent control of the RAN. It communicates with the O-gNB via the O-RAN E2 interface, which allows xApps to subscribe to RAN measurements and transmit control messages.

This Dockerized Near-RT RIC is connected to the O-gNB. It emulates a realistic O-RAN environment while satisfying latency constraints, with an approximate control loop delay of  $\sim 1$ s. The Near-RT RIC incorporates the SDL functionality, powered by an in-memory database (namely the Redis) to support real-time data sharing among xApps. The SDL acts as a publish/subscribe data broker, allowing any xApp to read from or write to keys in the shared Redis datastore. It serves as the internal database of the Near-RT RIC, maintaining live RAN data. This serves as the Near-RT RICs internal database for live RAN data. By leveraging Redis through the SDL functionality, our xApps achieve low-latency data exchange, with read and write operations for small metric sets completing within milliseconds.

### B. KPI MONITORING XAPP

A specialized xApp, implemented as a Python module `kpm_mon1.py` in [36], was developed to continuously collect key performance metrics (KPMs) from O-RAN. This KPI Monitoring xApp uses the standardized KPM E2SM to subscribe to periodic UE-level metrics from the O-gNBs E2 agent. Upon each E2 indication message containing the latest KPM reports, the xApp parses the metrics received and pushes them into the Redis database.

Each UE's report is stored under a unique key, typically indexed by UE ID, and is updated at every reporting interval, which is set to  $\sim 1$ s in our configuration. The use of Redis enables efficient storage and retrieval of these time-series metrics by other xApps operating within the Near-RT RIC in real time. In essence, the KPM Monitoring xApp acts as a bridge between the O-RAN E2 interface and the Near-RT RIC's internal database. This ensures that live O-RAN statistics are continuously accessible to xApps.

### C. ML-BASED DETECTION XAPP DESIGN

To detect flooding attacks in O-RAN, we implemented a detection xApp that ingests the live KPM stream from Redis and applies a deep learning model to classify each UE's state

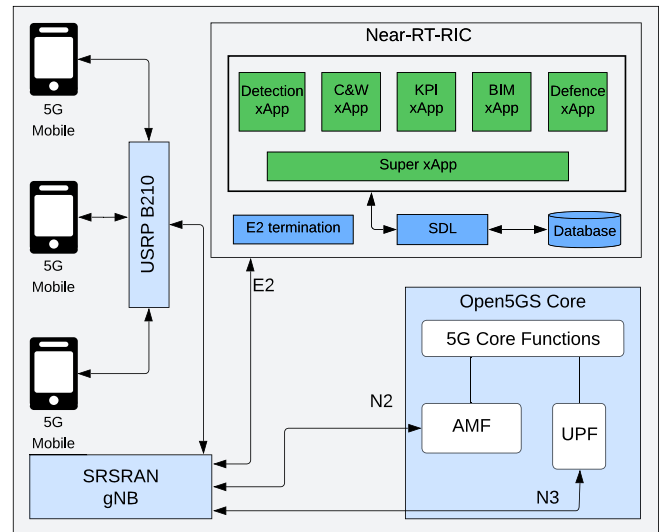


FIGURE 3. The illustration of the O-RAN testbed used for the implementation of the attacks and defense mechanisms.

as benign or under attack. The choice of features and model architecture was guided by domain knowledge of network attacks. We selected eight KPM features that capture critical indicators of network performance and resource usage under stress, including both radio-link and core-level metrics:

- *DRB.UEThpUl / DRB.UEThpDl*: Uplink and downlink data throughput per UE (reflecting traffic load).
- *RRU.PrbUsedUl / RRU.PrbUsedDl*: Uplink and downlink physical resource block (PRB) utilization (fraction of radio resources consumed).
- *DRB.RlcDelayUl*: Uplink RLC buffer delay (ms), indicative of queuing under congestion.
- *DRB.PacketSuccessRateUl*: Uplink packet success rate over the air interface (ratio of successfully delivered packets).
- *DRB.RlcSduToVolDl / DRB.RlcSduToVolUl*: Downlink and uplink RLC SDU volume transmitted (bytes), representing traffic volume at the RAN layer.

These features provide a multi-faceted view of each UEs connection. Sudden spikes or anomalies across these metrics can signal abnormal conditions, such as a malicious flood causing high PRB usage and low success rate. All feature values are normalized using Min-Max scaling to the range  $[0,1]$  to ensure stable ML processing.

**DNN Architecture & Training:** The detection model is a lightweight feed-forward DNN implemented in PyTorch. It has an input layer with eight neurons (one per feature), followed by three hidden layers and a single output neuron. The hidden layers use rectified linear unit (RELU) activations and dropouts for regularization. Specifically, the architecture follows:  $8 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$ , where the numbers indicate the number of neurons per layer. The first hidden layer includes 64 neurons and 30% dropout; the second has 32 neurons and 20% dropout; and the third includes

16 neurons. The output neuron uses a sigmoid activation function to represent the probability that UE is an attacker.

The model was trained offline using supervised learning. A binary cross-entropy loss function was minimized with the Adam optimizer using a learning rate of  $1 \cdot 10^{-3}$  and batch size of 64 for 100 epochs. All input features were normalized using the same scaler fitted on the training set.

**Training Data and Labeling:** To obtain realistic training data, we orchestrated controlled traffic scenarios on the testbed. Benign traffic was generated by UEs using iperf to create steady uplink data flows to a test server through the 5G core, simulating typical user behavior such as video streaming. Malicious traffic was produced by configuring a UE to perform periodic flooding attacks using hping3, generating high-rate internet control message protocol (ICMP) and user datagram protocol (UDP) packets targeting the user plane function (UPF) in the core network.

These volumetric denial-of-service (DoS) floods caused significant uplink traffic surges and network load. By alternating between attack and idle periods, we labeled the collected KPM time-series accordingly: metrics during attack periods were labeled as “malicious,” while benign periods and other UEs were labeled “benign.” The resulting dataset showed clear distinctions between the attack and normal behavior, such as spikes in uplink PRB usage and drops in packet success rate during attacks.

**Real-Time Inference Pipeline:** Once deployed in the Near-RT RIC, the detection xApp performs continuous inference on new KPM data every  $\sim 1$ s in a closed-loop cycle. The execution flow is as follows:

- 1) *Data Retrieval:* The xApp fetches the latest KPM vector for each UE from Redis, as stored by the KPI Monitoring xApp in the previous interval.
- 2) *Preprocessing:* The raw feature vector is normalized using the pre-fitted Min-Max scaler.
- 3) *DNN Inference:* The normalized vector is input into the DNN, producing a score  $s \in [0, 1]$  indicating the predicted likelihood of attack.
- 4) *Thresholding:* If  $s \geq \tau$ , UE is flagged as suspicious; otherwise, it is considered normal. We set the threshold  $\tau = 0.7$  to balance true positive rate and false alarms.

This inference loop completes in a few milliseconds per cycle and fits well within the  $\sim 1$ s latency budget of the Near-RT RIC. UEs producing scores up to certain values are classified as attackers in real-time and are logged for evaluation purposes. We empirically set the detection threshold to  $\tau = 0.7$ , which yielded a good balance of high true positive rate and low false alarm rate in preliminary tests. UEs with  $s \geq 0.7$  are thus classified as malicious in real-time. The entire inference for all active UEs completes in a few milliseconds per cycle, allowing the detection xApp to operate comfortably within the  $\sim 1$ s near-RT window.

#### D. ADVERSARIAL ATTACK XAPP (MALICIOUS XAPP)

In parallel with the detection xApp, we implemented a malicious xApp that launches adversarial ML attacks on the

KPM data stream. This xApp, functioning as an intelligent adversary, possesses white-box knowledge of the detection model and aims to deceive it by manipulating the KPM feature vectors stored in Redis.

The adversarial xApp continuously monitors the same UE metrics available to the detection xApp. When it detects a scenario in which a UE is likely to be flagged as malicious (i.e., the detection score is approaching the classification threshold), it intervenes by generating a specially crafted perturbation to the input features. These adversarial perturbations are designed to reduce the detector’s confidence while minimally altering the original data, thereby evading detection without introducing conspicuous anomalies.

Two well-known adversarial attack techniques were implemented in this framework for evaluation: the C&W attack and the BIM attack. Each attack alters the feature vector in real-time, immediately overwriting the original metrics in Redis before the detection xApp’s next inference cycle.

##### 1) C&W ATTACK XAPP

The C&W is an optimization-based adversarial attack that finds minimal input modifications sufficient to flip a classifier’s output. In our context, this attack targets the detection DNN’s decision function. For a given UEs normalized feature vector  $x$ , the adversarial xApp computes a perturbed vector  $x'$  that minimizes the distance from  $x$  while simultaneously causing the DNN to misclassify the input as benign.

The attack is formulated as an optimization problem, where the objective function combines an  $L_2$ -norm penalty and a misclassification loss term:

$$L(x') = \|x' - x\|_2^2 + c \cdot f(x'), \quad (4)$$

where  $f(x')$  is a penalty term that is large when the DNN correctly classifies  $x'$  as an attack (thus the optimizer drives the model toward misclassifying  $x'$  as benign), and  $c$  is a regularization weight. We set  $c = 10$  and use the Adam optimizer to iteratively adjust  $x'$  for up to 300 steps with a step size of 0.05.

To maintain real-time operation, the adversarial xApp does not attack every sample. Instead, it continuously monitors the detection score for each UE. If a UE’s raw DNN output score  $s$  exceeds a safety margin (e.g.,  $s \geq 0.7$ ), indicating a high likelihood of detection, the C&W routine is triggered for that UE. The resulting adversarial feature vector  $x'$  is then inverse-normalized to the original KPM scale and written back to Redis, overwriting the original metrics for that UE before the detection xApp processes it in the next cycle. By injecting this imperceptibly perturbed data into the shared pipeline, the adversarial xApp effectively misleads the detector without raising suspicion.

In practice, the C&W attack proved highly effective at degrading detection performance. The DNN’s output score for attacked UEs dropped from high confidence levels (e.g.,  $s \approx 0.9$ ) to below 0.2, which prevented them from being flagged. This manipulation resulted in an overall detection

accuracy drop from approximately 92% in clean conditions to just 16% under active C&W attack.

## 2) BIM ATTACK XAPP

In addition to the C&W attack, we implemented a gradient-based adversarial attack using the BIM. This attack is an extended multi-step variant of the FGSM that perturbs the input in the direction of the gradient of the loss function. These perturbations are applied iteratively in small steps to gradually increase the model's misclassification probability.

At each iteration, a small fixed perturbation (with a step size of 0.01 in normalized feature space) is added to each selected feature in the direction of the gradient's sign. The result is then clipped to ensure all features remain within valid bounds. For each triggering event, we ran BIM for 10 iterations. We explored several variants of the BIM attack to balance stealth and effectiveness:

- *Full-feature BIM*: Perturbs all eight selected KPM features in each iteration.
- *Inverted-direction BIM*: Applies some perturbations in directions that contradict expected attack behavior (e.g., increasing a metric that normally decreases during an attack) to better mimic benign conditions.

In real time, the adversarial xApp applies the BIM procedure whenever a UE's detection score approaches the classification threshold. The resulting adversarial vector is immediately written back to Redis, replacing the original KPM metrics before the detection xApp performs. Because each BIM step introduces only a minor change, the cumulative distortion remains low. Despite the minimal perturbation, BIM attack consistently fooled DNN. Detection confidence for the attacker UEs dropped to near zero, suppressing alarms. In some cases, altered feature distributions even caused model to misclassify benign UEs, due to shifting global patterns. Overall, BIM attack significantly degraded detection reliability. The detector's accuracy dropped from 0.92% (no attack) to 0.10 under BIM conditions, indicating substantial loss in precision and recall.

## E. CLOSED-LOOP EXECUTION AND LATENCY

Figure 3 illustrates the timing of our closed-loop system, which integrates the KPM Monitoring xApp, the detection xApp, and the adversarial xApp. Each 1-second control cycle begins with the KPM Monitoring xApp writing fresh UE-level KPM metrics to Redis. Immediately afterward, the adversarial xApp (if active) intercepts this data and perturbs the metrics for selected UEs, completing its optimization within a few tens of milliseconds. Finally, the detection xApp reads the (potentially modified) data from Redis and performs inference to identify any ongoing attacks.

This pipeline is carefully synchronized using Redis publish-subscribe notifications, ensuring that the adversarial overwrite (if triggered) occurs before the detection xApp fetches the latest metrics. Measurements showed that even the more computationally intensive C&W attack executed

in approximately 50 ms–100 ms on average while the BIM attack completed in less than 10 ms. As a result, the entire detection-and-response loop remains well within the  $\sim 1$  s Near-RT RIC latency budget.

This real-time dynamic creates an interaction: as soon as the detector recognizes anomalous behavior, the adversarial xApp rapidly distorts the evidence, poisoning the shared data used for inference. The consequence is that even a high-performing ML detector can be undermined in practice when its input data is manipulated just before prediction. While the detection xApp maintained high accuracy on clean data, the carefully constructed adversarial examples generated by the malicious xApp significantly reduced its effectiveness. These results highlight the critical need for more robust, adaptive, and secure ML-based mechanisms in O-RAN architectures to counter adversarial manipulation in real-time environments.

## F. IMPLEMENTATION OF MITIGATION XAPP

The mitigation strategy is implemented within a real-time xApp designed to secure UEs against adversarial threats. The implementation operates sequentially through multiple integrated detection mechanisms, ensuring accurate anomaly identification and responsive mitigation.

Initially, the xApp continuously retrieves KPMs data from the centralized "Super xApp" using RESTful API calls. Each received JavaScript Object Notation (JSON)-based data report undergoes a hashing process using SHA-256 to verify uniqueness, thereby eliminating redundant processing by identifying and discarding duplicate data. Following the uniqueness verification, essential network performance indicators are extracted from the UE measurement data. Extracted features are standardized using a pre-trained scaler to ensure consistent comparison across data samples.

The first detection mechanism implemented is statistical anomaly detection using Z-score analysis. For every UE, the system maintains a sliding window of the last 20 feature observations. Each incoming measurement is statistically compared against historical data within this sliding window. If two or more features significantly deviate beyond a predefined Z-score threshold, the measurement is flagged as anomalous.

The second mechanism involves ensemble machine learning inference. The xApp performs multiple forward passes through a pretrained neural network classifier for each measurement, generating individual probability predictions. These predictions are averaged to derive a robust confidence level. A measurement is flagged as anomalous if the averaged confidence level exceeds a predefined threshold.

Additionally, the implementation integrates gradient norm heuristics by calculating the gradient magnitude of the neural network output with respect to input features. Persistent instances of unusually low gradient magnitudes, averaged over recent observations, indicate possible adversarial manipulation, prompting an anomaly detection flag.

The fourth mechanism is a feature delta check, which measures immediate changes in feature values compared to

the previous measurement. A sudden cumulative feature shift exceeding a predefined threshold automatically triggers an anomaly flag, ensuring rapid detection of abrupt attacks.

To consolidate anomaly detection and minimize false positives, temporal majority voting is applied. Each UE maintains a historical log of the last five anomaly detection outcomes. The final decision to execute a suppression action requires anomalies to be consistently identified—specifically, a suppression action is triggered only when anomalies are detected in at least three of the five most recent evaluations, reflecting consistent anomalous behavior over multiple observations.

When persistent anomalous behavior indicative of an attack is detected, the xApp responds via the E2 interface by issuing control requests to reduce the PRB allocated to the identified UE, effectively isolating the threat. To prevent repetitive suppression actions against the same UE, the implementation includes a cooldown mechanism, enforcing a five-second delay before another suppression action can be issued for the same UE.

Runtime profiling of the defense xApp indicates an average total latency of 76 milliseconds per KPI report. This includes 32 ms for ensemble DNN inference, 18 ms for gradient norm analysis, 12 ms for Z-score based statistical anomaly detection, and 14 ms for feature delta analysis and temporal voting. All detection modules run sequentially within the xApp and remain well under the 1000 ms real-time constraint of Near-RT RIC operations. These measurements confirm the defense xApp is computationally efficient and suitable for deployment in live O-RAN environments.

## VI. RESULTS AND EVALUATION

In this section, we present the results of the experiments conducted on our O-RAN testbed. These experiments evaluate the baseline performance of the detection xApp, the impact of adversarial evasion attacks, and the effectiveness of the mitigation xApp. The detection xApp (a DNN model) was deployed on the RIC and monitored per UE KPI reports in real time. C&W and BIM attack modules were integrated via a Super xApp controller to manipulate incoming KPI data. Mitigation xApp was implemented to analyse anomalies and enforce mitigations, as detailed in the implementation report. Below, we first discuss the classification performance of the detection xApp, then the effects of adversarial attacks, and finally the improvements achieved with the mitigation xApp.

### A. DETECTION XAPP BASELINE PERFORMANCE

The DNN-based detection xApp achieved a high level of overall accuracy in distinguishing between benign and malicious traffic under normal (attack-free) conditions. Specifically, the model obtained an overall classification accuracy of 92.37% on the test dataset. As illustrated in Figure 4, precision, recall and the F1-score are shown for each class, with benign classes indicated by a value of 0.0 and malicious classes indicated by a value of 1.0. It is evident that benign traffic is identified with a high degree

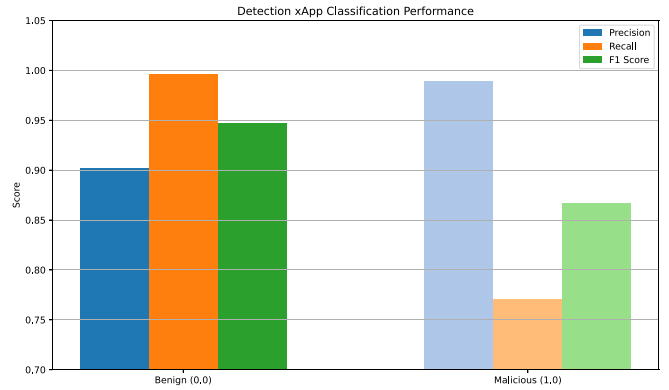


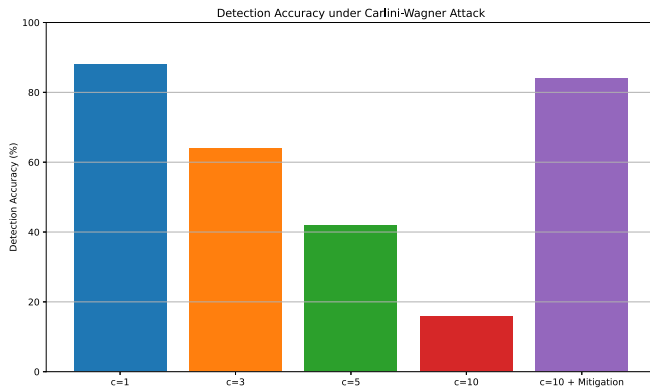
FIGURE 4. Precision, recall, and F1-score of the detection xApp for benign and malicious UEs under normal conditions.

of precision and recall (both approximately 0.90%–0.98%), while the malicious class is also detected with a high degree of reliability, with an F1-score of 0.8664 (approximately 0.98% precision and 0.77% recall). This minor discrepancy in the malicious class performance is indicative of a number of false alarms, which is to be expected given that the detection threshold was calibrated to strike a balance between detecting attacks and avoiding false positives. Overall, the detection xApp demonstrated robust baseline performance. The establishment of baseline metrics is imperative in order to establish a point of reference prior to the introduction of adversarial attacks.

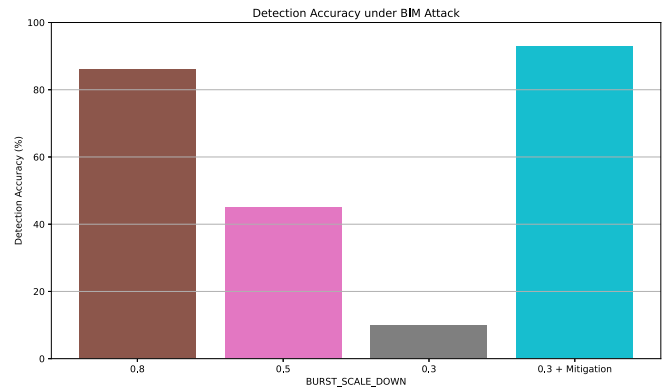
**Role of the detection xApp:** In our architecture, the detection xApp serves as a real time anomaly detector that flags misbehaving or malicious UEs based on their KPI patterns. In practice, this means the xApp can alert the RIC or trigger mitigation actions whenever a UE exhibits suspicious behavior. The high baseline accuracy (92%) indicates that under normal conditions the system can reliably distinguish malicious activity from benign traffic, which is crucial for maintaining network performance and security. If the detection accuracy were to drop significantly, many malicious behaviors would go unrecognized potentially allowing an attacker to consume network resources or disrupt service without triggering any alarms. Thus, strong baseline detection performance is critical to ensuring that the network can proactively respond to threats and avoid degradation of service quality due to undetected malicious UEs.

### B. IMPACT OF ADVERSARIAL ATTACKS

In order to evaluate the robustness of the detection xApp, a series of tests were conducted in which the xApp was subjected to two adversarial evasion attacks that had previously been implemented as xApps: a C&W gradient-based attack and a BIM feature-suppression attack. The attack xApps intercept real-time KPI reports via the Super xApp and perturb the data in an attempt to evade detection. The attack intensity was then increased incrementally, and the resulting detection accuracy was measured. In the absence of any defensive measures, both attacks caused a substantial



**FIGURE 5.** Detection accuracy under Carlini-Wagner attacks with increasing strength. Mitigation restores accuracy at high attack levels.



**FIGURE 6.** Detection accuracy under BIM attacks. Strong suppression lowers accuracy, but mitigation restores it effectively.

deterioration in the performance of the detection xApp, as outlined below.

### 1) C&W MALICIOUS ATTACK XAPP

The C&W attack module generates small adversarial perturbations to the KPI feature vector with the objective of minimizing the DNN's confidence in detecting an attack. The efficacy of this attack was evaluated at varying perturbation levels, controlled via the  $CW\_loss\_parameter$   $c$ . As the value of  $c$  increased, the detection accuracy exhibited a substantial decline.

As shown in Figure 5, the detection accuracy dropped from 88% at  $c = 1$  (minimal perturbation), to 64% at  $c = 3$ , 42% at  $c = 5$ , and 16% at  $c = 10$ . These results confirm that high-magnitude C&W perturbations can effectively blind the baseline detector, reducing its accuracy to as low as 10%.

The C&W attack operates by subtly manipulating KPI values to simulate benign activity, while remaining within plausible statistical ranges allowing it to evade the detection thresholds of the original DNN model. Notably, the integration of the proposed defense xApp significantly restored lost detection capability. When the mitigation xApp was activated, the detection accuracy at  $c = 10$  improved from 16% (without defense) to 84%, as indicated in Figure 5.

In summary, the defense system proved effective in identifying and countering C&W based adversarial attacks that were not detected by the baseline model. It restored detection accuracy to near-nominal levels, even in the presence of severe perturbations. While lower perturbation levels ( $c = 1-5$ ) were less detrimental to performance (accuracy ranging from 42% to 88% without defense), the result at  $c = 10$  highlights the critical importance of deploying the mitigation xApp under worst-case evasion attempts.

### 2) BIM MALICIOUS ATTACK XAPP

The Basic Iterative Method (BIM) attack was also evaluated, wherein selected KPI features were scaled down or perturbed to mimic a "low activity" UE and evade detection. During the experiments, the BIM attacker specifically targeted bursty KPI features by applying a reduction factor, denoted as

$BURST\_SCALE\_DOWN$ , along with additive noise. The goal was to observe the response of the detection xApp under various suppression intensities.

Figure 6 summarizes the detection accuracy across three BIM attack settings of increasing severity, where a lower  $BURST\_SCALE\_DOWN$  represents a more stealthy malicious UE profile. With  $BURST\_SCALE\_DOWN = 0.8$  (i.e., a modest decrease in feature values), the detection accuracy remained relatively high at 86%, indicating that the model still identified most anomalies. When the suppression factor was reduced to  $BURST\_SCALE\_DOWN = 0.5$ , the detection accuracy dropped to 45%. In the most aggressive case, with  $BURST\_SCALE\_DOWN = 0.3$  and noise sampled from the range  $[2.0, 5.0]$ , the detector's accuracy deteriorated to just 10%. This worst-case BIM scenario effectively blinded the baseline detector, a phenomenon similar to the outcome observed with the C&W attack, and consistent with expectations that BIM can reduce model performance to 20% or lower.

It is evident that the mitigation xApp has had a significant impact on the outcomes, demonstrating a substantial enhancement in results. As demonstrated in Figure 6, the implementation of the defense mechanism for the most severe BIM attack (0.3 scale) enhanced the accuracy from a mere 10% to 93%. In essence, the defense system was able to restore detection to almost perfect levels within that particular scenario. It is anticipated that the defense mechanism will be effective against moderate BIM attacks. However, for the 0.8 and 0.5 scale cases, it was observed that the baseline accuracy (86% and 45%, respectively) was maintained in the absence of defense. This suggests that the mitigation mechanisms would ensure high accuracy in the presence of perturbations, with the exception of the most subtle variations. The primary conclusion of this study is that, with the implementation of adversarial defense mechanisms, the detection application demonstrated notable resilience to the BIM attacker's feature suppression tactics. This resilience was evidenced by the mitigation of an attack that had initially resulted in 90% of malicious events going undetected, as reflected by the recovery of 93% accuracy.

### C. MITIGATION XAPP EFFECTIVENESS

The results presented above demonstrate the efficacy of the mitigation xApp in countering both types of adversarial attacks. Deploying our defense mechanisms led to near-complete recovery of the detection performance, even in the face of the most potent evasion attempts. For the C&W attack at  $c = 10$ , the detection accuracy improved from 16% (no defense) to 84% with the defense activated a 68 percentage point increase. In the context of the BIM attack with 0.3 scaling, the accuracy jumped from 10% to 93%, representing an 83 percentage point increase. These substantial improvements effectively restored the system to its nominal accuracy range and validate the effectiveness of the proposed defense strategies. In practice, the defence xApp reduced the percentage of detectable malicious events from approximately 80 – 90% (when attacked without defence) to less than 10% in our tests, ensuring the RAN remained secure and stable even while attacks were occurring. The mitigation xApp’s design combining statistical anomaly detection, ensemble DNN inferences, and adversarial behavior checks underpins these results. In real time, the defense xApp monitors each UE’s metrics for abnormal temporal patterns and other signs of evasion, complementing the primary DNN classifier. Specifically, our implementation uses a sliding window of recent observations and a majority vote mechanism to reduce false positives. For each UE, the defense xApp examines a moving window of the last few KPI reports (using Z-scores to detect outliers in the sequence) and aggregates alerts over that window. We empirically chose a window size of 5 observations for this temporal voting mechanism, as it provided a good balance between detection speed and accuracy. A shorter window (e.g., 3 observations) was found to trigger more false positives from temporary KPI fluctuations (since even a brief benign spike could be mistaken as an anomaly), whereas a much longer window (e.g., 10 observations) would introduce a larger detection delay (an ongoing attack would need to persist longer before the system flags it) and could allow the adversary more time to cause damage. By using a 5 sample window for majority voting, the defense xApp ensures that an anomaly must persist over several consecutive reporting intervals to be considered malicious, which significantly cuts down on false alarms due to momentary noise while incurring only a modest detection delay. In practice, this means an attacker’s behavior needs to remain consistently abnormal for a short duration (5 cycles) to trigger mitigation a reasonable trade off that caught all the sustained attacks in our tests without overreacting to single sample deviations. Once suspicious behavior has been flagged by multiple detection indicators within the window, the system reclassifies what would otherwise be false negatives as likely attacks. This multi layered temporal voting approach proved crucial in achieving robust resilience against the evaluated attacks it allowed the defense xApp to filter out random fluctuations and focus on persistent adversarial patterns. The outcome, as evidenced by the

restored accuracy figures, is that the Near-RT RIC with our defense is able to quickly detect and neutralize adversarial manipulations in real time, even those subtle enough to fool the baseline DNN.

While our current implementation uses a feedforward DNN model as the core detector, the design of the mitigation xApp is model agnostic and can be adapted to other ML architectures. The anomaly detection and gradient-sensitivity mechanisms can be tailored to sequence-based models like RNNs, Long Short-Term Memorys (LSTMs) and Gated Recurrent Units (GRUs) by incorporating recurrent hidden state monitoring and time step gradient analysis. For transformer based xApps, adversarial behavior can be flagged by analyzing entropy shifts in attention distributions or sudden changes in output confidence. These generalizable strategies make the defense xApp adaptable to various xApp types beyond UE anomaly detection, including traffic classification, resource optimization, or mobility management.

In summary, our findings suggest that while the detection xApp performs well under typical conditions, adversarial attacks can significantly compromise its performance if left unmitigated. However, by integrating the mitigation xApp’s defenses, the Near-RT RIC can proactively detect and neutralize these attacks, maintaining high detection accuracy and network stability even in the face of sophisticated adversarial behavior. These results underscore the importance of adversarial resilience in RAN control applications.

### VII. CONCLUDING REMARKS AND FUTURE OUTLOOK

In this paper, we demonstrated that ML-driven xApps within O-RAN are susceptible to hostile exploitation. We proved that potent evasion assaults, illustrated by the C&W and BIM methods, can significantly impair an xApp’s model correctness and compromise its functioning. Our real-time investigations validated that such attacks markedly diminished the detection xApp’s accuracy, consistent with previous reports of substantial performance declines under adversarial perturbations. To counter this danger, we created a dynamic defense xApp that incorporates many techniques (statistical anomaly detection, ensemble DNN inference, gradient-based heuristics, etc.) to detect and neutralize hostile inputs in real time. The proposed defense xApp was prototyped in a Near-RT RIC and assessed in real time. Empirical observations indicated that while adversarial attacks significantly compromised the original xApp’s detection capabilities, the implementation of our defense mechanism largely reinstated the model’s performance to near baseline levels. For instance, when under the strongest attacks, our defense restored detection accuracy from 16% back up to 84% for the C&W attack, and from 10% up to 93% for the BIM attack, effectively nullifying the adversaries impact. This restoration of accuracy during attacks highlights the effectiveness of a multifaceted, adaptive defense strategy within O-RAN.

These findings carry important implications for the design of a secure and resilient O-RAN architecture. They highlight the need to treat AI/ML components in the RIC as mission

critical elements that must be protected against sophisticated manipulation. The open and modular nature of O-RAN, while enabling innovation, expands the attack surface and calls for security by design principles. Our results suggest that incorporating defensive xApps into the Near-RT RIC can provide an additional layer of protection, continuously monitoring model outputs and data integrity to quickly detect anomalies. Moreover, the necessity of such defenses aligns with a broader shift toward zero-trust principles in network architecture. A zero trust architecture (ZTA) for O-RAN would ensure that no xApp or internal component is implicitly trusted without verification, employing multi layered controls to safeguard confidentiality, integrity, and availability. Embracing this paradigm—“never trust, always verify”—at the architectural level can significantly strengthen the resilience of O-RAN deployments against both external and insider threats.

In the future, there are several promising directions for extending this work. First, the realism of the evaluation environment should be enhanced by incorporating live network traffic and multi-vendor components into the testbed. This will ensure that defense mechanisms are validated under conditions mirroring real-world O-RAN deployments. Second, the scalability of the proposed defenses must be investigated in large-scale deployments - it is crucial to verify that a Near-RT RIC can orchestrate anomaly detection across many nodes and xApps without introducing prohibitive latency or overhead. Third, advanced learning techniques such as federated learning and continual model training offer a way for the system to adapt to evolving threats. For example, a collaborative federated approach could enable multiple RICs or operators to share insights about new attack patterns without exposing raw data, while continuous learning would allow each model to update itself over time as new adversarial tactics emerge. Finally, a comprehensive end-to-end security strategy should be pursued by integrating full-stack zero-trust principles across all O-RAN layers. This entails embedding stringent authentication, authorization, and fine-grained monitoring at every layer and interface of the O-RAN, thereby holistically fortifying the O-RAN ecosystem against both external intrusions and insider threats. By following these directions, future O-RAN networks can achieve a higher level of robustness and trustworthiness, which can ensure that the benefits of open, intelligent O-RAN architectures are not undermined by security vulnerabilities.

## ACRONYM LIST

|      |                                    |
|------|------------------------------------|
| 3GPP | 3rd Generation Partnership Project |
| 5G   | fifth-generation                   |
| AI   | artificial intelligence            |
| API  | application programming interface  |
| DoS  | denial of service                  |
| BIM  | basic iterative method             |
| C&W  | Carlini–Wagner                     |
| DDoS | distributed-denial-of-service      |

|              |                                                                    |
|--------------|--------------------------------------------------------------------|
| DoS          | denial-of-service                                                  |
| DNN          | deep neural network                                                |
| E2E          | end-to-end                                                         |
| E2SM         | E2 service model                                                   |
| FGSM         | Fast Gradient Sign Method                                          |
| gNB          | next generation node B                                             |
| GRU          | Gated Recurrent Unit                                               |
| IDS          | intrusion detection system                                         |
| ICMP         | internet control message protocol                                  |
| JSON         | JavaScript Object Notation                                         |
| KPI          | key performance indicator                                          |
| KPM          | key performance metric                                             |
| LSTM         | Long Short-Term Memory                                             |
| MFA          | multi-factor authentication                                        |
| ML           | machine learning                                                   |
| NG-RAN       | next-generation radio access network                               |
| Near-RT RIC  | near-real-time RAN intelligent controller                          |
| Non-RT RIC   | non-real-time RAN intelligent controller                           |
| O-RAN        | open radio access network                                          |
| O-RAN SC RIC | O-RAN Software Community Near-Real-Time RAN Intelligent Controller |
| O-CU         | open centralized unit                                              |
| O-DU         | open-distributed unit                                              |
| O-RU         | open-radio unit                                                    |
| O-gNB        | open-next generation node B                                        |
| O-Cloud      | open cloud                                                         |
| PGD          | projected gradient descent                                         |
| PRB          | physical resource block                                            |
| QoS          | quality of service                                                 |
| RAN          | radio access network                                               |
| RIC          | RAN intelligent controller                                         |
| RNN          | recursive neural network                                           |
| RELU         | rectified linear unit                                              |
| SMO          | service management and orchestration                               |
| SDL          | shared data layer                                                  |
| SS           | secure slicing                                                     |
| UE           | user equipment                                                     |
| UDP          | user datagram protocol                                             |
| UPF          | user plane function                                                |
| USRP         | universal software radio peripheral                                |
| VNF          | virtual network function                                           |
| ZTA          | zero trust architecture                                            |

## REFERENCES

- [1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, “Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges,” *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1376–1411, 2nd Quart., 2023.
- [2] K. Alam et al., “A comprehensive tutorial and survey of O-RAN: Exploring slicing-aware architecture, deployment options, use cases, and challenges,” 2024, *arXiv:2405.03555*.
- [3] W. Azariah, F. A. Bimo, C. W. Lin, R. Cheng, N. Nikaein, and R. Jana, “A survey on open radio access networks: Challenges, research directions, and open source approaches,” *Sensors*, vol. 24, no. 3, p. 1038, 2024.
- [4] P. Porambage, M. Christopoulou, B. Han, M. Asif Habibi, H. Bogucka, and P. Kryszkiewicz, “Security, privacy, and trust for open radio access networks in 6G,” *IEEE Open J. Commun. Soc.*, vol. 6, pp. 332–361, 2025.

- [5] M. A. Habibi et al., "Toward an open, intelligent, and end-to-end architectural framework for network slicing in 6G communication systems," *IEEE Open J. Commun. Soc.*, vol. 4, pp. 1615–1658, 2023.
- [6] M. S. Wani, M. Kretschmer, B. Schröder, A. Grebe, and M. Rademacher, "Open RAN: A concise overview," *IEEE Open J. Commun. Soc.*, vol. 6, pp. 13–28, 2025.
- [7] M. A. Habibi, F. Z. Yousaf, and H. D. Schotten, "Mapping the VNFs and VLS of a ran slice onto intelligent pops in beyond 5G mobile networks," *IEEE Open J. Commun. Soc.*, vol. 3, pp. 670–704, 2022.
- [8] M. A. Habibi, G. M. Yilma, U. Fattore, X. Costa-Pérez, and H. D. Schotten, "Unlocking O-RAN potential: How management data analytics enhances SMO capabilities?" *IEEE Open J. Commun. Soc.*, vol. 5, pp. 4710–4730, 2024.
- [9] *O-RAN Work Group 1 Use Cases and Overall Architecture: AIML-Security-Analysis*, O-RAN ALLIANCE e.V., Alfter, Germany, 2024.
- [10] M. A. Habibi, G. M. Yilma, X. Costa-Pérez, and H. D. Schotten, "Unifying 3GPP, ETSI, and O-RAN SMO interfaces: Enabling slice subnets interoperability," in *Proc. IEEE FNWF*, Baltimore, MD, USA, 2023, pp. 1–8.
- [11] P. K. Kakani, H. Djuitcheu, and H. D. Schotten, "Evaluation of xApps and their security in next-generation open radio access networks (ORAN)," Authorea Preprints, 2024.
- [12] "O-RAN work group 11 (security work group): Study on security for near real time RIC and xApps," O-RAN ALLIANCE e.V., Alfter, Germany, Rep. TR-0-R003-v05.00, 2024.
- [13] T. Dayaratne et al., "Exploiting and securing ML solutions in near-RT RIC: A perspective of an xApp," 2024, *arXiv:2406.12299*.
- [14] *O-RAN Work Group 11 (Security Work Group): O-RAN Study on Security for Artificial Intelligence and Machine Learning (AI/ML) in O-RAN*, O-RAN ALLIANCE e.V., Alfter, Germany, 2024.
- [15] M. A. Habibi, B. Han, M. Saimler, I. L. Pavón, and H. D. Schotten, "Towards an AI/ML-driven SMO framework in O-RAN: Scenarios, solutions, and challenges," in *Proc. IEEE FNWF*, 2024, pp. 1–8.
- [16] "O-RAN work group 11 (security work group): AIML-security-analysis," O-RAN ALLIANCE e.V., Alfter, Germany, Rep. O-R004-v02.00, 2024.
- [17] J. Groen et al., "Implementing and evaluating security in o-ran: Interfaces, intelligence, and platforms," *IEEE Netw.*, vol. 39, no. 1, pp. 227–234, Jan. 2025.
- [18] S. B. Mallikarjun, M. Asif Habibi, M. Dixit, X. Costa-Pérez, M. Debbah, and H. D. Schotten, "Exploring y1 communications and services in O-RAN: Background, privacy, and security," *IEEE Open J. Commun. Soc.*, vol. 6, pp. 4638–4666, 2025.
- [19] M. M. Qazzaz, Ł. Kulacz, A. Kliks, S. A. Zaidi, M. Dryjanski, and D. McLernon, "Machine learning-based xapp for dynamic resource allocation in o-ran networks," 2024, *arXiv:2401.07643*.
- [20] C.-F. Hung, Y.-R. Chen, C.-H. Tseng, and S.-M. Cheng, "Security threats to xApps access control and E2 interface in o-ran," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 1197–1203, 2024.
- [21] N. Sapavath et al., "Experimental study of adversarial attacks on ml-based xapps in o-ran," in *Proc. IEEE Global Commun. Conf.*, 2023, pp. 6352–6357.
- [22] M. Kouchaki, J. Moore, M. Zhang, and V. Marojevic, "Advancing O-RAN security: Integrated intrusion detection and secure slicing xApps," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2024, pp. 1–2.
- [23] A. S. Abdalla, J. Moore, N. Adhikari, and V. Marojevic, "ZTRAN: Prototyping zero trust security xApps for open radio access network deployments," *IEEE Wireless Commun.*, vol. 31, no. 2, pp. 66–73, Apr. 2024.
- [24] P. Trakadas et al., "A cost-efficient 5G non-public network architectural approach: Key concepts and enablers, building blocks and potential use cases," *Sensors*, vol. 21, no. 16, p. 5578, 2021.
- [25] M. K. Motalleb et al., "Towards secure intelligent O-RAN architecture: Vulnerabilities, threats and promising technical solutions using LLMs," 2024, *arXiv:2411.08640*.
- [26] H. Djuitcheu et al., "Exploring the implications and methodologies of securing the E2 interface," Authorea Preprints, 2024.
- [27] A. F. Diallo and P. Patras, "Sabre: Cutting through adversarial noise with adaptive spectral filtering and input reconstruction," in *Proc. IEEE Symp. Security Privacy (SP)*, 2024, pp. 2901–2919.
- [28] S. Cresci, M. Petrocchi, A. Spognardi, and S. Tognazzi, "Adversarial machine learning for protecting against online manipulation," *IEEE Internet Comput.*, vol. 26, no. 2, pp. 47–52, Mar./Apr. 2021.
- [29] H. Waghela, J. Sen, and S. Rakshit, "Robust image classification: Defensive strategies against FGSM and PGD adversarial attacks," 2024, *arXiv:2408.13274*.
- [30] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Security Privacy (SP)*, 2017, pp. 39–57.
- [31] N. Carlini and D. Wagner, "Adversarial examples are not easily detected: Bypassing ten detection methods," in *Proc. 10th ACM Workshop Artif. Intell. Secur.*, 2017, pp. 3–14.
- [32] T. R. Sarkar et al., "Evaluating adversarial robustness: A comparison of FGSM, Carlini-Wagner attacks, and the role of distillation as defense mechanism," 2024, *arXiv:2404.04245*.
- [33] M. G. Abdu-Aguye, W. Gomma, Y. Makihara, and Y. Yagi, "Detecting adversarial attacks in time-series data," in *Proc. IEEE Int. Conf. Acoustics, Speech Signal Process. (ICASSP)*, 2020, pp. 3092–3096.
- [34] Z. Zhao et al., "Attack as detection: Using adversarial attack methods to detect abnormal examples," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 3, pp. 1–45, 2024.
- [35] P. Rathore, A. Basak, S. H. Nistala, and V. Runkana, "Untargeted, targeted and universal adversarial attacks and defenses on time series," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2020, pp. 1–8.
- [36] "KPI monitoring xApp overview." O-RAN Software Community. 2023. [Online]. Available: <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-app-kpimon/en/latest/overview.html>



**PRUDHVI KUMAR KAKANI** received the B.Sc. degree in mechanical engineering from the Lakireddy Balireddy College of Engineering, India, in 2016, and the M.Sc. degree in commercial vehicle technology from the Rheinland-Pfälzische Technische Universität (previously known as Technische Universität Kaiserslautern), Germany, in 2022. He is currently pursuing the Ph.D. degree with the Division of Wireless Communications and Radio Navigation, Rheinland-Pfälzische Technische Universität, Germany, where he has been working as a Research Fellow since September 2023. From 2022 to 2023, he worked as a DevOps Engineer for ASAP. His main research interests include xApp development, open RAN, near-RT-RIC, security, machine learning, and radio access networks.



**MOHAMMAD ASIF HABIBI** received the B.Sc. degree in telecommunications engineering from Kabul University, Afghanistan, in 2011, and the M.Sc. degree in systems engineering and informatics from the Czech University of Life Sciences, Czech Republic, in 2016. He is currently pursuing the Ph.D. degree with the Division of Wireless Communications and Radio Positioning, University of Kaiserslautern-Landau, Germany, where he has been working as a Research Fellow since January 2017. From 2011 to 2014, he worked as a Radio Access Network Engineer for HUAWEI. His main research interests include network management, network slicing, machine learning, and network architecture.



**MANJUNATH REDDY CHAVVA BALANNAGARI** received the B.Sc. degree in computer science and engineering from Sathyabama University, Chennai, in 2020. He is currently pursuing the M.Sc. degree in informatik with the Rheinland-Pfälzische Technische Universität (previously known as Technische Universität Kaiserslautern), Germany. His primary research interests encompass xApp development, open RAN, near-real-time RIC, security, machine learning, and radio access networks.



**XAVIER COSTA-PÉREZ** (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in telecommunications from the Polytechnic University of Catalonia, Barcelona. He is the Head of the 6G Networks R&D, NEC Laboratories Europe, a Scientific Director with the i2Cat R&D Center, and a Research Professor with ICREA. His team contributes to products roadmap evolution as well as to European Commission R&D projects and received several awards for successful technology transfers. In addition, the

team contributes to related standardization bodies: 3GPP, ETSI NFV, ETSI MEC, and IETF. He has been a 5GPPP Technology Board member, served on the Program Committee of several conferences (including IEEE Greencom, WCNC, and INFOCOM), published at top research venues, and holds several patents. He was the recipient of the National Award for his Ph.D. thesis. He also serves as an Editor for the IEEE TRANSACTIONS ON MOBILE COMPUTING and IEEE TRANSACTIONS ON COMMUNICATIONS journals.



**HANS D. SCHOTTEN** (Member, IEEE) received the Ph.D. degree from the RWTH Aachen University of Technology, Germany, in 1997. From 1999 to 2003, he worked with Ericsson. From 2003 to 2007, he worked with Qualcomm. He became a Manager of a R&D Group, a Research Coordinator for Qualcomm Europe, and the Director for Technical Standards. In 2007, he accepted the offer to become a Full Professor with the University of Kaiserslautern, where he served as a Dean with the Department of Electrical

Engineering from 2013 to 2017. In 2012, he became the Scientific Director with the German Research Center for Artificial Intelligence and the Head of the Department for Intelligent Networks. Since 2018, he has been a Chairman of the German Society for Information Technology and a member of the Supervisory Board of VDE.